Software Engineering

Spring 2026

Henry
VILLAR

USE CASES. REQUIREMENTS AND SPECIFICATIONS

Personas, Use Cases, Requirements, Specifications, Priorities, High-Level Mock-up, Storyboards

Week 1 Summary

1. What is SE?
2. Why is SE important?
3. What will we cover in this course?
4. How will you be evaluated?
5. What is the next due date for our project?

Summary. Cont.

1. Software Engineering (SE) is “The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software”.
2. SW Technology change often. Principles and Best Practices of SE change little. Close to real life experience.
3. SE Process & Methodologies | Requirements and Specifications | Usability and UI Design Principles | QA & Testing | SE Project Management, Teamwork & IP
4. M0 (10pts), M1-5 (50pts), and Final Exam (40pts)

M0 due next week Tuesday February 10th , by 11:59PM

4.2 Grading of M0

This assignment is worth 10% of your class grade, as indicated in the syllabus. The grade for this assignment will be determined according to the following criteria:

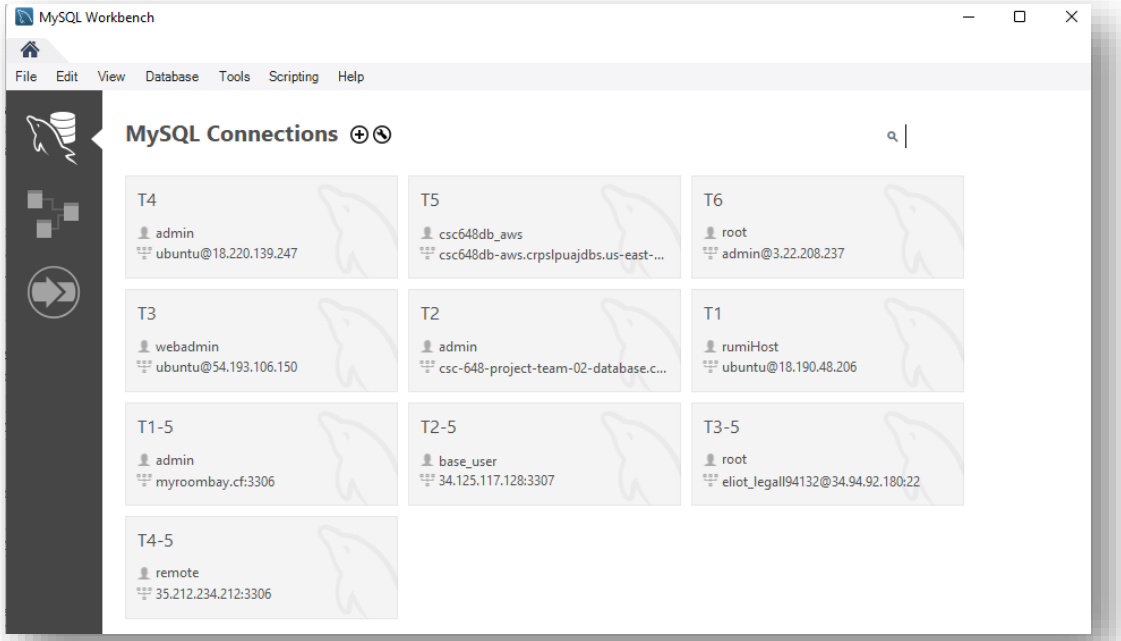
Category	Points
Correctly Installed and Configured Software Stack	2
Correctly Populated Credentials Folder in GitHub & correct use of GitHub	3
Correct team WWW page functionality, deployment and proper usage of team's Software Stack for creating web page	3
Individual SE Dev Environment Video uploaded in Canvas	2
Total:	10

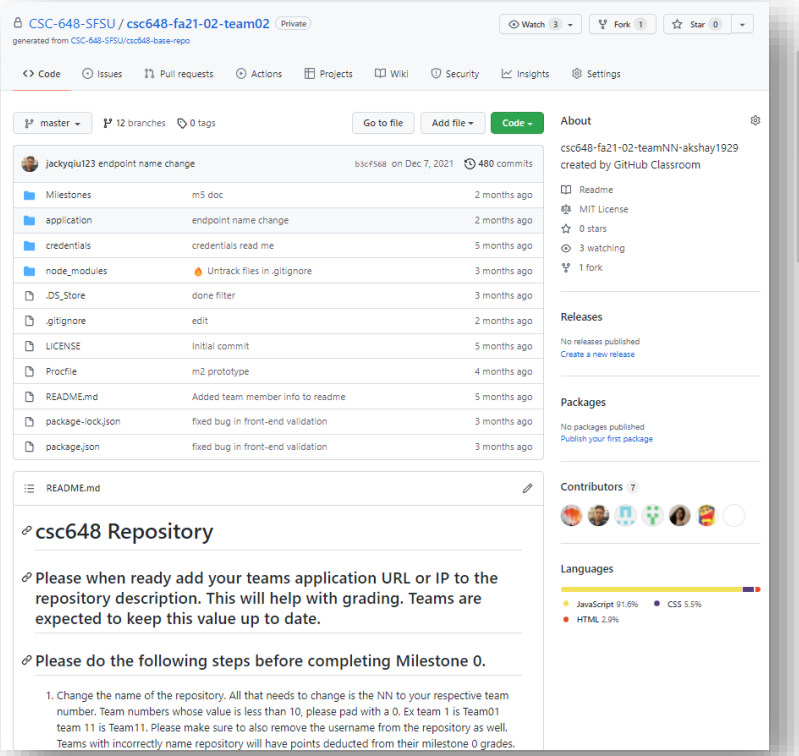
*"When your team has decided on a software stack, the **team lead** needs to email me detailing your software stack. The email must have the format indicated in the M0 document. The due date is tonight by 11:59PM."*

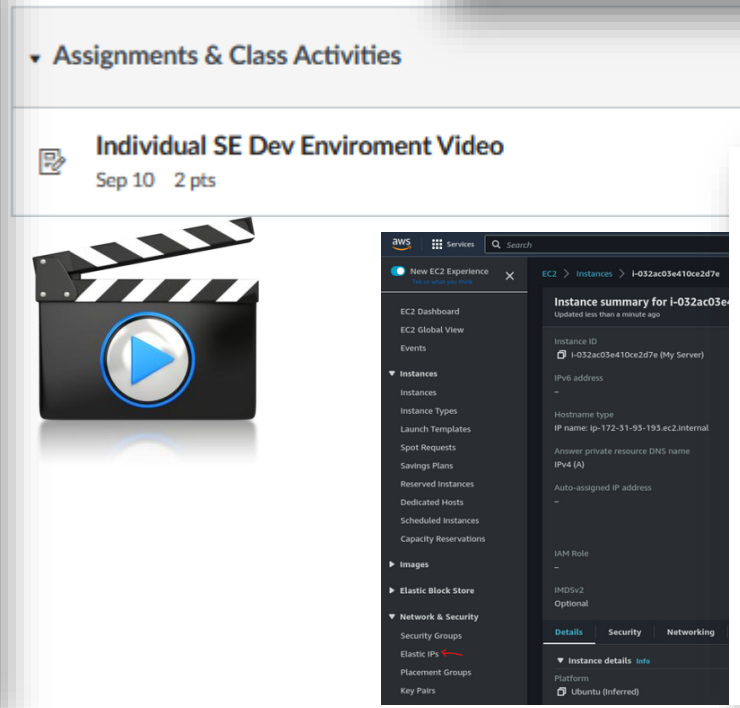
M0 Expectation

All projects will be inspected and graded on the due date. The key for grading will be your team's:

1. Correct installation and configuration of software stack,
2. Correct setup and usage of GitHub, and the
3. Deployment of your About Me Web Application
4. Individual video demonstrating config-connection







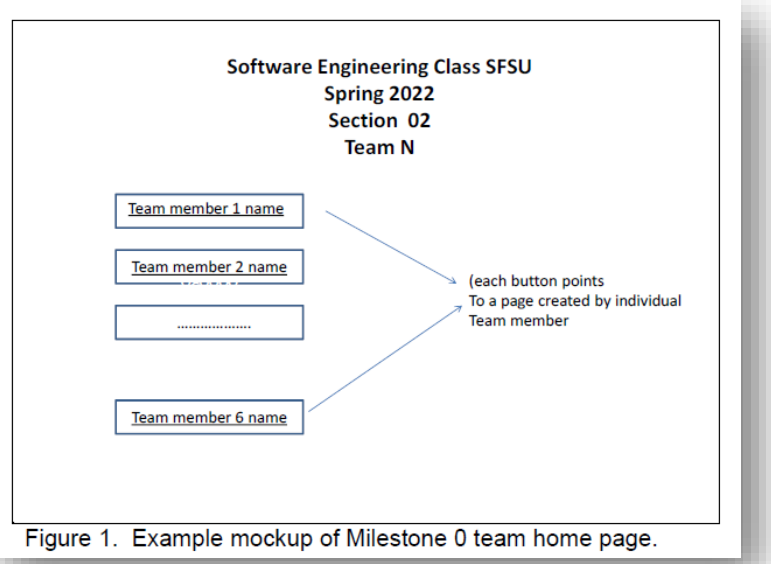


Figure 1. Example mockup of Milestone 0 team home page.

Today’s Topics

- Personas
- Use Cases
- Requirements
- Specifications
- Priorities
- High-Level Mock-up, Storyboards

Software Engineering (SE) Course CSC 648-848
Spring 2026
High Level Description of Class Team Project
Web App: AI LifeOS – Intelligent Student Co-Pilot Platform

Henry Villar
San Francisco State University

Acknowledge
Collaboration and team project content is based on material created by Dr. D. Petkovic & Anthony Souza

Introduction

This document describes at a **very high-level** the software engineering (SE) team **Final Project,** a Web application that student teams will develop this semester. Each student team will design and develop its own version of the same web application which is described in this report. Using only this brief high-level description to initiate the project is on purpose somewhat vague in order to engage students in the full cycle of SW application development starting from scratch and leading to delivery of a final application.

Selection and options for SW development and application deployment tools to be used will be described in the class and in Milestone 0.

Final project organization polices and rules of participation, as well as grading rubrics are described in the class slides about final team project, posted on Canvas. The final team project carries **50% of the class grade**, which each milestone contributes to the final score. In general, all team members get the same grade for the final project (exceptions will be explained in the class, team members who do not participate/collaborate would have less points). Full and maximum focus and participation/attendance to team project are required from each team member.

Student teams are assigned based on academic records and other methods in order to properly mix the skills and experience. Each team will have a formal ***team lead, front and back end engineer, github lead/master and other roles according with their needs and member skills***. Each team will communicate with instructors mainly via their team lead.

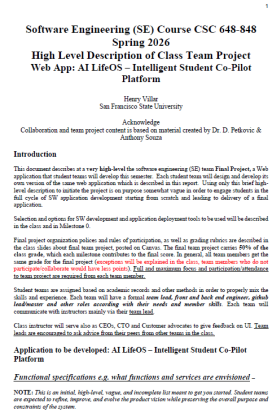
Class instructor will serve also as CEOs, CTO and Customer advocates to give feedback on UI. Team leads are encouraged to ask advice from their peers from other teams in the class.

Application to be developed: AI LifeOS – Intelligent Student Co-Pilot Platform

Functional specifications e.g. what functions and services are envisioned –

NOTE: *This is an initial, high-level, vague, and incomplete list meant to get you started. Student teams are expected to refine, improve, and evolve the product vision while preserving the overall purpose and constraints of the system.*

Class Team Project:
AI LifeOS – Intelligent Student Co-Pilot Platform



Software Engineering (SE) Course CSC 648-848
Spring 2026
High Level Description of Class Team Project
Web App: AI LifeOS – Intelligent Student Co-Pilot Platform

Henry Villar
San Francisco State University

Acknowledge
Collaboration and team project content is based on material created by Dr. D. Petkovic & Anthony Souza

Introduction

This document describes at a very high-level the software engineering (SE) team Final Project, a Web application that student teams will develop this semester. Each student team will design and develop its own version of the same web application which is described in this report. Using only this brief high-level description to initiate the project is on purpose somewhat vague in order to engage students in the full cycle of SW application development starting from scratch and leading to delivery of a final application.

Selection and options for SW development and application deployment tools to be used will be described in the class and in Milestone 0.

Final project organization policies and rules of participation, as well as grading rubrics are described in the class slides about final team project, posted on Canvas. The final team project carries 50% of the class grade, which each milestone contributes to the final score. In general, all team members get the same grade for the final project (exceptions will be explained in the class, team members who do not participate/collaborate would have bad peers.). Full and minimum focus and participation/attendance in team project are required from each team member.

Student teams are assigned based on academic records and other methods in order to properly mix the skills and experience. Each team will have a formal *scrum lead, front and back end engineer, github lead/master and other roles according with their needs and member skills*. Each team will communicate with instructor mainly via their team lead.

Class instructor will serve also as CEOs, CTO and Customer advocates to give feedback on UI. Team leads are encouraged to ask advice from their peers from other teams in the class.

Application to be developed: AI LifeOS – Intelligent Student Co-Pilot Platform

Functional specifications e.g. what functions and services are envisioned –

NOTE: *This is an initial, high-level, vague, and incomplete list meant to get you started. Student teams are expected to refine, improve, and evolve the product vision while preserving the overall purpose and constraints of the system.*

The system will allow students to create a **personal dashboard** where they can manage goals, projects, learning activities, and personal development areas such as academic success, career preparation, leadership, wellness, and life skills.

A core component of the platform will be a **powerful search and discovery engine** that enables users to find and explore:

- Goals and project templates
- Productivity and study workflows
- Academic and career resources
- AI tools and learning aids
- Campus-related opportunities and services
- Public community-shared content

Users will be able to **search by keywords and filter** results by multiple criteria such as:

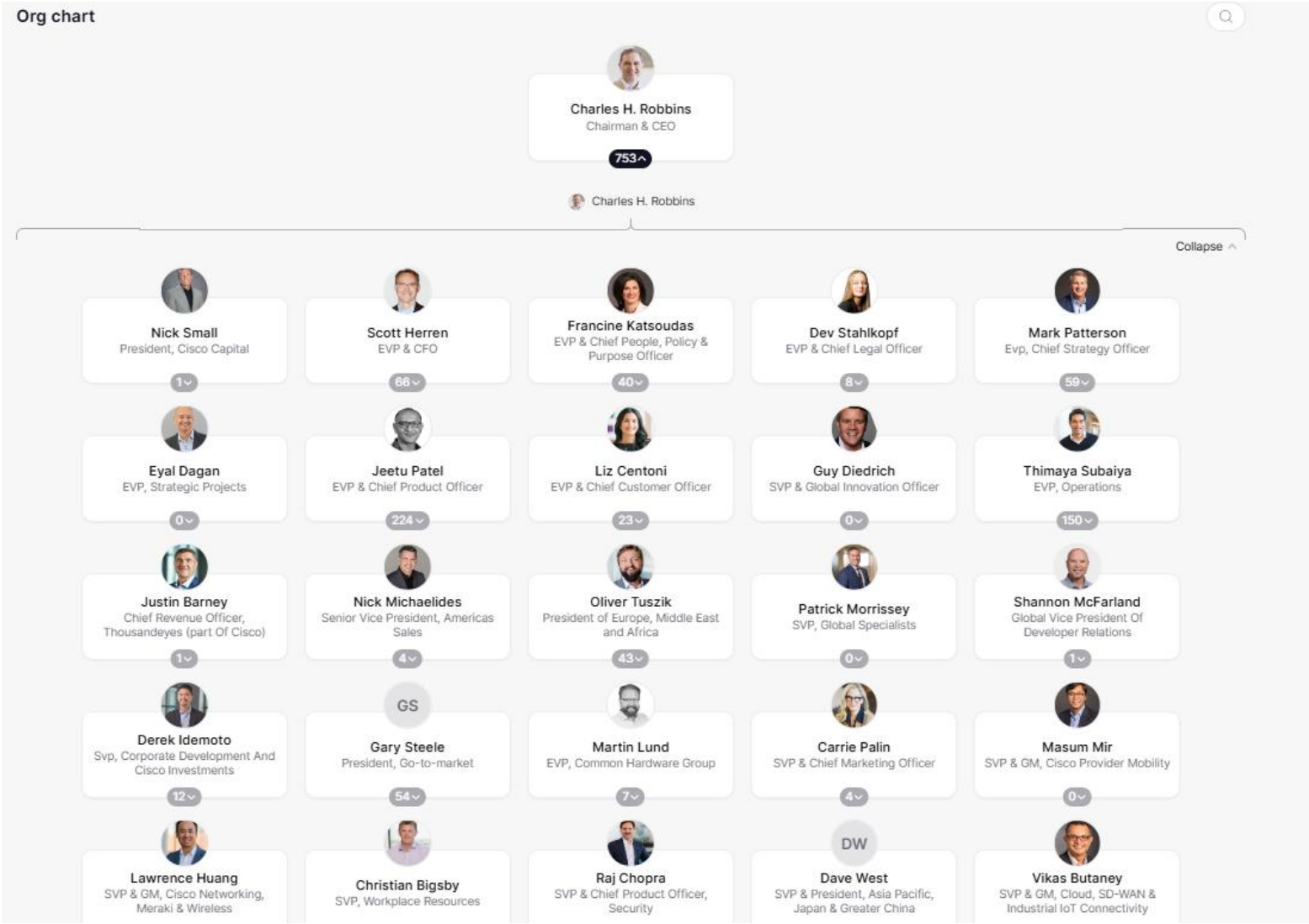
- Category (academic, career, wellness, leadership, finance, creativity, research, etc.)
- Use case (studying, job preparation, mental wellness, skill building, productivity, project management)
- Content type (tools, templates, workflows, reflections, resources, events)
- Skill area (technical, soft skills, leadership, research, health, communication)
- AI-enabled vs. non-AI resources
- Free vs. paid (informational only, no transactions)

The system will provide **interactive dashboards**, visual timelines, charts, and personalized reports.

- The platform will support community-oriented capabilities, such as:
- Browsing and searching public goal templates and workflows
- Sharing resources, tools, and strategies
- Discovering popular or trending student tools
- Rating, tagging, and bookmarking content
- In-site messaging between users (webmail only)

The platform targets three primary audiences:

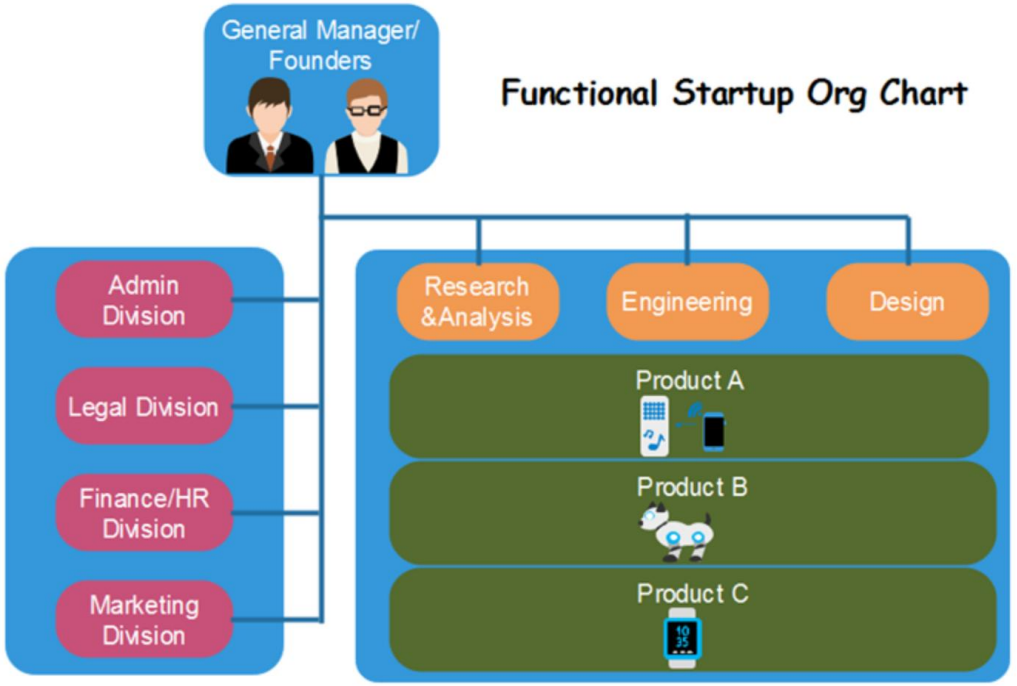
- **College Students** – Primary users managing academics, career preparation, and personal development.
- **Faculty & Academic Programs** – Secondary users interested in ethical AI use, learning support, and student development tools.
- **University Support Organizations** – Career centers, tutoring centers, and student success programs that may showcase categorized resources.

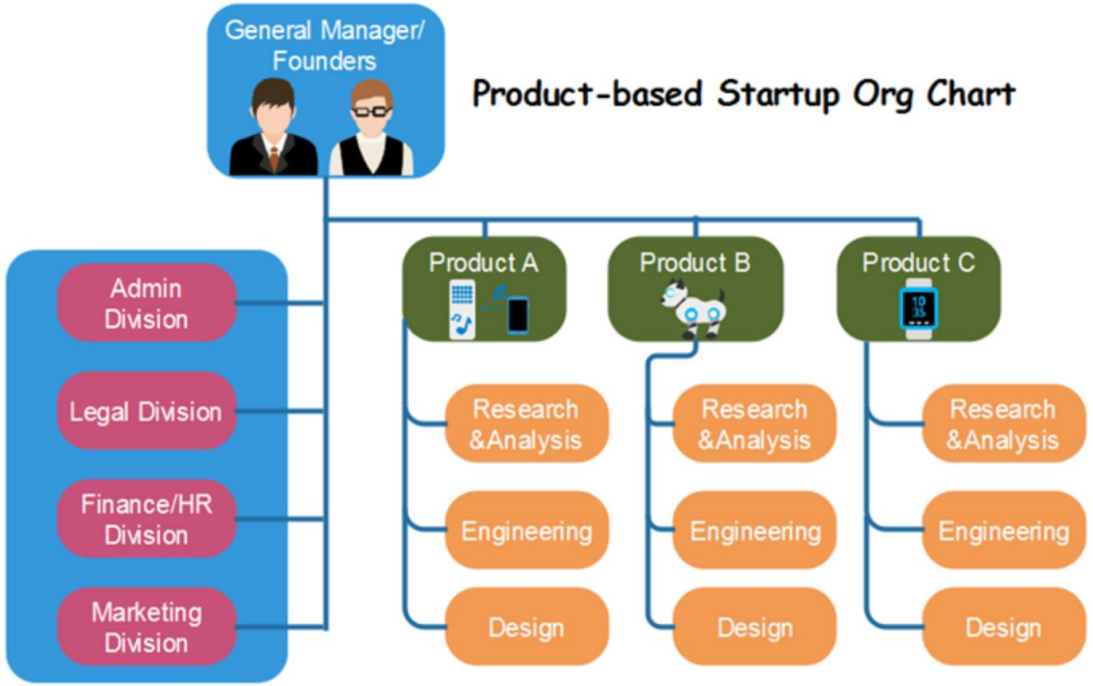


Corporate Organization

Source: https://theorg.com/org/cisco

Startup Organization





Source: https://www.orgcharting.com/org-chart-successful-startup/

Important

SW Engineering Process Components

- There are several basic components present in all SW Engineering methodologies:
  1. *SW Requirements and Specification*: functionality, constraints, look and feel, GUI, performance
  2. *SW Design and Implementation*
  3. *SW Validation*: formal, test plans, QA
  4. *SW Maintenance and Evolution*: ongoing changes (and fixes) to meet customer needs

  These components are applied in different way and order (linear, iterative, spiral…) depending on chosen SW Engineering methodology or models (I.e. *waterfall, extreme, agile* etc.)

Use Cases, Requirements, Specifications

Important

Iterations!!!!!

- Personas and Use Cases: describe who are the main actors ("*personas*") and then a story ("*use cases*") of *WHAT services/functions users will need*, in plain English. Focus on WHAT not HOW. Involve marketing, sales, competitive study, focus groups
- Requirements: developed from use cases, usually by marketing, engineering, sales (*focus on WHAT not how*):
  - Functional (what functions the SW has, what does it do)
  - Non-functional – what properties the SW has (security, privacy, performance, deploy in Cloud? etc.)
- Specifications – more concrete engineering guidance of HOW to develop the SW (often combined with requirements into SW requirement specifications - SRS)
- Followed by architectural design, implementation, testing in an *iterative fashion*

Personas – a way to represent user types for your use cases and UI design – and testing

- From Interaction Design Organization, downloaded 05/28/18
  - https://www.interaction-design.org/literature/article/personas-why-and-how-you-should-use-them
- *Personas are fictional characters, which you create based upon your research in order to represent the different user types that might use your service, product, site, or brand in a similar way.* ***Creating personas will help you to understand your users' needs, experiences, behaviors and goals.*** *Creating personas can help you step out of yourself. It can help you to recognize that different people have different needs and expectations, and it can also help you to identify with the user you're designing for. Personas make the design task at hand less complex, they guide your ideation processes, and they can help you to achieve the goal of creating a good user experience for your target user group.*

Important

Personas

- Personas have:
- Attitude
- Skills
- Limitations
- Pain points
- Goals
- Personas go over scenarios (use cases) using your application in the appropriate context (home, on the road…)
- Create one page persona description (with image) to be handy during the design – some examples below
- https://www.google.com/search?q=ui+design+persona&hl=en&source=lnms&tbm=isch&sa=X&ved=0ahUKEwjmu_Wv6KjbAhUfIDQIHTHTD2YQ_AUICigB&biw=1440&bih=782

Example persona: SFSU students

Add some image to make it more personal

Freshmen

Transfer

International

Graduate

Counselors

Events

Source: https://future.sfsu.edu/admissions/

- About [any student above, let's call her Mary from the CS department]:
  - Very busy (school + perhaps a side job)
  - Passionate for new tech trends
  - Loves community/sports and technology
  - Environmentally conscious
  - Participate in justice, diversity and equality activities and events
  - Has advanced WWW skills
- Goals and scenario:
- Current job is not related to technology and want to start entering her professional field
- No professional network, limited knowledge of the companies in the Bay Area, and overwhelmed from the Jobs portal (LinkedIn and others)

Personas vs. use cases

- Persona: *general* characteristics of the users related to your applications

- Use cases: *What services and functions* personas will use your envisioned system

Important

Use Cases

- They are major driver for requirements and specs and architecture
- (the other driver is business/client needs)

- Design systems to satisfy use cases (e.g. user needs) not the other way around

- Think of the user first, and also of client/business

- http://en.wikipedia.org/wiki/Requirements_analysis

Important

Use cases

- One of the most powerful methods. *Must do!*
  - Define **actors** in the application – **persona** – a model of a typical user e.g. define user skill level, pain points….
  - Describe **user tasks** and errors
    - Know frequency and importance of tasks
  - Describe user **environment and context**
  - Describe or use data items form data glossary – <u>be consistent</u>

- Methods for gathering information:
  - Ask users (single or team)
  - Observe users in their actual environment
  - Look at user documentation or any other information
  - Surveys, focus groups
  - Ask your own marketing, sales, support, client reps
  - Read WWW, product reviews
  - Ask your friends

Important

Use Cases >> Task analysis

- Task name, description and current duration as well as frequency of usage
- User skill level, experience and role
- Work steps including usage of other systems (printers, faxes…)
- System features, GUI screens currently used
- Difficulties and errors in using current system
- Current systems performance and reliability
- Current user satisfaction and “pain points”
- Follow-up questions
- Sample inputs, screens etc.

- Good to compare current and future (with your system) use cases in terms of number of steps, time to complete etc.

Important

Use cases levels of detail

- *High level* – focus on WHAT the user does at high level (no details of HOW)
- *Lower-level* – more details, spell out functions, steps etc. (<u>less used</u>)

- High-level use cases are critical
  - Drive them by user needs NOT the constrains of the envisioned system. Compromise only if you must, be a user advocate.
  - Focus on WHAT user needs not HOW to design or implement

Important

Usage of use cases

- Communicates vision of the system easily understood to non-technical people (CEO, marketing, sales, investors)
- Drives the design and requirements
- Use to test all aspects of the system, especially the UI
- Use to develop QA plans

Examples of bad high level use case, then a corrected one

Important

- Mary wants to search for an entry tech job. She goes to our site, types "cybersecurity" in the yellow search button which is on the upper right corner of the site, hits the GO button then goes to the list of search results presented at the bottom of the screen in two columns
- Too many UI design details (underlined) – it is too early for this!
- Better one (leaves UI design details for later)
- Mary wants to search for an entry tech job. She goes to our site and searches by skills then reviews the list of search results

Use cases even at high level can drive UI design – e.g. lazy registration

- Mary wants to search for a cybersecurity job on our site. She chooses the appropriate function from our www site, and only gets one result. She is prompted to register or log in to see more results
- This way you ensure lazy registration implementation metaphor where user is prompted to register/login only when already using the function – this is the right way
  - https://mockitt.wondershare.com/ui-ux-design/ui-design-patterns.html

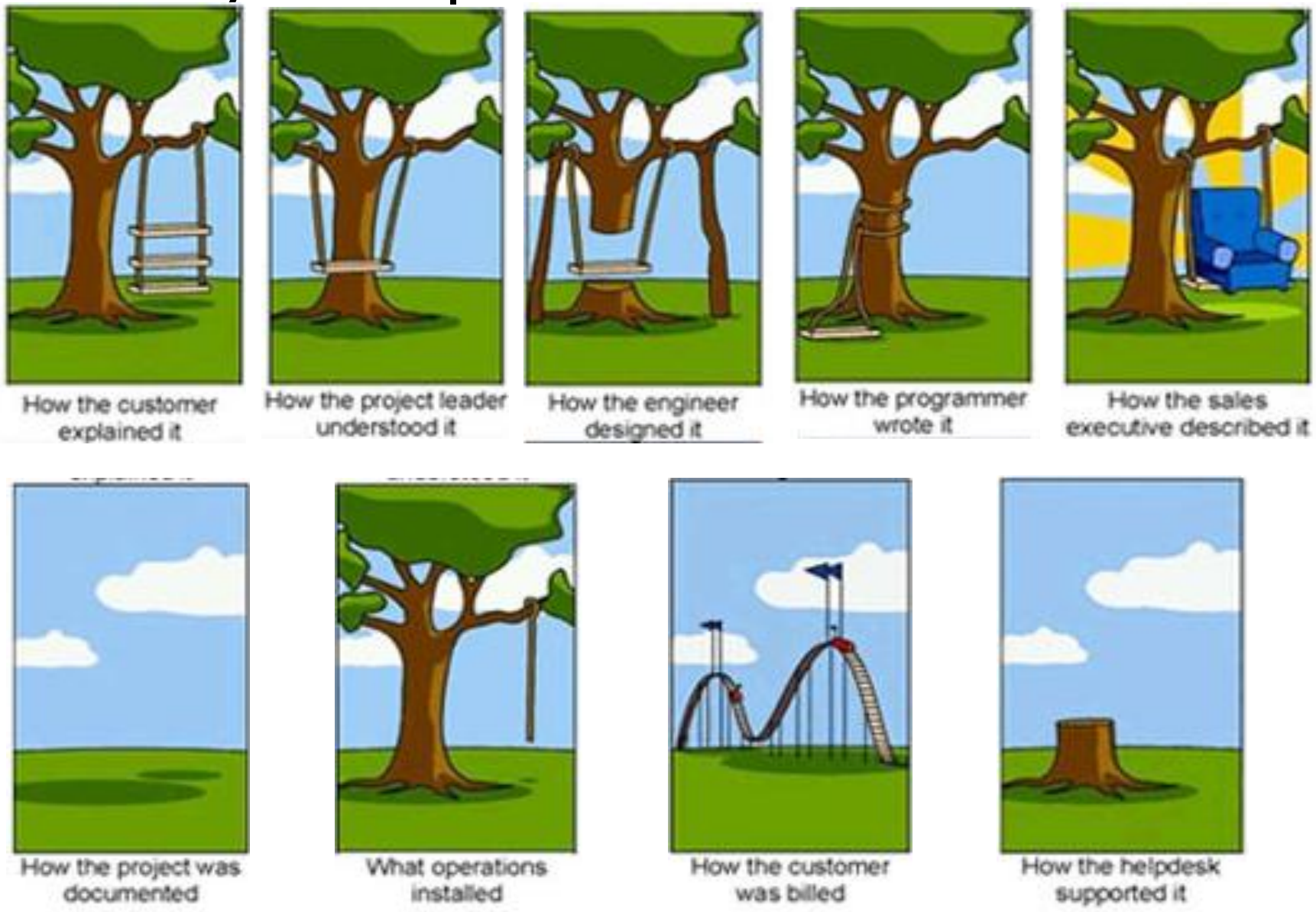
Use Cases, Requirements, Specifications

Important

Iterations!!!!!

- **Personas and Use Cases**: describe who are the main actors ("*personas*") and then a story ("*use cases*") of *WHAT services/functions users will need*, in plain English. Focus on WHT not HOW. Involve marketing, sales, competitive study, focus groups
- **Requirements**: developed from use cases, usually by marketing, engineering, sales (*focus on WHAT not how*):
  - **Functional** (what functions the SW has, what does it do)
  - **Non-functional** – what properties the SW has (security, privacy, performance, deploy in Cloud? etc.)
- **Specifications** – more concrete engineering guidance of HOW to develop the SW (often combined with requirements into SW requirement specifications - **SRS**)
- Followed by **architectural design, implementation, testing in an** ***iterative fashion***

Why Requirements?



Important

Software Requirements

- Descriptions of a SW system – starting point for specifications, design and implementation
- One of the main and early component in any SE process – focus is on WHAT not HOW
- Main vehicle of communication between:
  - Idea initiator and the rest of the world
  - Clients and SW providers
  - Customer and product development (SW, marketing, sales)
  - Product development and management
  - Product development and marketing
  - Between developers (especially at different locations)
  - Developers and QA
- Good overview http://en.wikipedia.org/wiki/Requirements_analysis

Important

What are Requirements

- Requirements: Abstract statements or descriptions of SW services, functions or constraints

- For high-level requirements focus is on WHAT not HOW
- Level of abstraction goes from high (usually called requirement definitions) to more detailed (usually called requirement specifications)

Format and types of Requirements

Formalism

- Implicit, not written, “in your head only” - usually SW done for your own needs (e.g. your homework; self-education; fun)
- Verbal understanding and communication (extreme programming, small team, early prototype, team project at school)
- A few pages, free format (startups, small company, school)
- Structured documents (commercial, format often fixed and specific to a company)
- Very formal and elaborate (commercial, medical, avionics, government)
- Essential to ensure all have the same understanding of what needs to be done
- And, they change in time!!!!

Req. Categories: Functional and Non-functional Requirements

Important

- Functional requirements
  - WHAT DOES IT DO? Services and functions system provides. How will system react and behave depending on inputs. Specific functions provided to the user. Key data items
- Non-Functional requirements
  - HOW DOES IT DELIVER? Various constrains on timing, cost, ease of use, standards, security, performance, capacity, HW/SW environment, privacy etc.
- In some cases: **Domain requirements** (in case of certain app domains)
  - Comes from application domain and reflect specifics of that domain
  - Key in mutidisciplinary projects e.g. bioinformatics

Functional and Non-functional Requirements

Important

Functional requirements:

- *Functions:* Describe functions and services that system provides
  - Also*: Data description:* describe key data items and entities in the system (non compulsory but extremely useful)

Non-functional requirements:

- *System requirements:* describe the system requirements (architecture, system services, networks, platforms etc.)
- *Usability requirements*: describe specific usability and UI issues, users who will use the system, delivery clients
- *Performance requirements*: describe system performance (speed, accuracy, latency, delay, bandwidth etc.)
- *Storage, security, environmental requirements*
- *Marketing, legal requirements* (logos, branding, licensing)
- *Content* (size, formats…)
- *Privacy* (what data is collected, how is it used…)

Important

Functional Requirements

1) *Functions*: Describe functions and services provided by the system

Use SHALL
Not: will, may, should…

- Examples:
  - Users <u>shall</u> be able to search using image categories
  - Users shall be able to purchase music files using credit card
  - Music shall be accessible by author and title

Note: may contains some usability requirements (in choices of functions available), but usability can be addressed also in non-functional requirements too)

Functional requirements

Important

2) *Data description*: Describe key entities and data items and give them unique and meaningful names

- Main actors/users in the system (admin, customer…) and their roles
- Key data elements (book item, shopping cart, registration, search metadata). Important to describe *digital content* (see next slide)
- For each have the name, sub-items, means of input/initialization, usage, storage etc.

- Use these names consistently for documentation, UI, reference, naming of classes and variables, database schema

- Proper data description is the key for communication and understanding between/among developers and clients

Examples of High-level Requirements

- System shall provide full functionality without using mouse
- System shall provide user access to summary as well as raw data
- User shall be able to edit and correct the data after it is submitted
- System shall provide access to all data items based on customer name and account number
- Customer transactions shall be executed via secure internet connection
- Registration form: required for users to register. Contains name and e-mail, and optionally address, phone, affiliation. Stored in the database.
- Privacy: Data is used only by this WWW site for tracking the customer usage and shall not be shared with anybody else. Registered users may be contacted withy questions and promotions.

Important

Functional requirements are prioritized

- Details will be covered later but the key is to prioritize functions
  - Helps project planning
  - Helps UI design – high priority functions deserve more prominent UI exposure

Important

Non-Functional Requirements

- Refer to system properties and constraints such as:
  - Reliability
  - Response time
  - HW and networking requirements
  - Usability requirements
  - Marketing, legal, licensing
  - Media content (formats, size…)
  - Privacy: what is the data collected, how is the data used
  - Compatibility (e.g. which browsers…)

- Can refer to:
  - Product (product behavior like speed, reliability)
  - Organization (e.g. process, standards used)
  - External factors (e.g. branding, legal disclaimers displayed)

As important as functional requirements.

Non-Functional Requirements (2)

- Examples:
  - System shall response visually within 5 seconds
  - File size in no time shall exceed 2 Mbytes
  - Users with high-school diploma, after 1 hour training, shall complete the task in 5 minutes with no more than 2 errors.
  - Tools from XYZ shall be used for requirement management
  - Each requirement shall have identifiable portion of code associated with it referenced by module name and code line number
  - Each WWW page shall have official company logo in upper left corner
  - The following user data shall be colleted…; the data shall be used ONLY for for ….

Important

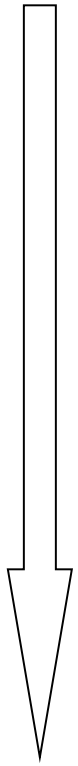
Non-functional requirements (3)

- Determined often by nontechnical constraints:
  - Business needs
  - Legal, marketing, copyright
- Not prioritized – they are all required
- Can not be changed solely by eng. team - change requires all stakeholders

Important

Another classification of requirement categories

Low detail



High level of detail

- *High-level requirements or vision or user requirements:* unstructured, natural language with some diagrams
- *System requirements or more detailed requirements*: more structured description of system functions and services. Can serve as a contract between client and developer. Natural language, structured language, simple mockups, data flow diagrams, charts
- *Software specifications*: more detailed SW description to serve as basis for design. Used by developers. Natural language, block diagrams, formulae, program design language (PDL), flow charts, data flow diagrams, mockups, structured language, database schemas etc.

(Exact formats and naming conventions often dictated by company policies)

Requirements: who reads them

Less technical detail (less technical readers)

- *User or High-level Requirements*: clients, users, client engineers, contract managers, systems architects, legal, marketing, executives
- *System requirements*: users, client engineers, system architects, SW developers
- *Software Design Specs*: System architects, SW developers, maybe client SW developers, users (for UI)

More technical detail (more technical readers)

Important

High-Level requirements: how should they be written?

- High-level: focus on WHAT not HOW. Write from “outside-in” perspective
- Leave design decisions (HOW) for later
- Use SHALL for must do, SHOULD for optional
- One concept per requirement
- Cover all necessary issues
- Avoid jargon
- Number them for tracking
- Group by priorities then by actors/subsystyems
- Talk in positive logic
- Details added as we go to lower-level requirements
- Not appropriate high level requirements:
  - Function X shall be provided by two buttons in upper left corner, and system should be fast.
  - Why?

Important

WHAT vs. HOW

- WHAT: what do you want, what the users want, what is the ultimate goal…➔ used for high level requirements, specs
- HOW: how to do this, how to design this ➔ for lower level, more detailed specs, design, implementation
- Ideally, first focus on WHAT then on HOW
- WHY?
- What happens if you do HOW too early?

Requirement measures

- As much as possible, <u>and where it applies</u>, put objectively measurable "tests" to verify that the requirement is fulfilled
  - "System should be easy to use" is OK as high level requirement from CEO, but without more details does not help design and QA.
- Need to add more details at next level, e.g.:
  - 80% of novice users to complete the process in 5 minutes with no more than 2 errors after 1 hour of training

Still WHAT and not HOW

Requirements measures (2)

Property	Measure
Speed	Processed transactions/second User/Event response time Screen refresh time
Size	K Bytes Number of RAM chips
Ease of use	Training time Number of help frames
Reliability	Mean time to failure Probability of unavailability Rate of failure occurrence Availability
Robustness	Time to restart after failure Percentage of events causing failure Probability of data corruption on failure
Portability	Percentage of target dependent statements Number of target systems

From: Ian Sommerville

Use Cases, Requirements, Specifications

Important

Iterations!!!!!

- **Personas and Use Cases**: describe who are the main actors (“*personas*”) and then a story (“*use cases*”) of *WHAT services/functions users will need*, in plain English. Focus on WHT not HOW. Involve marketing, sales, competitive study, focus groups
- **Requirements**: developed from use cases, usually by marketing, engineering, sales (*focus on WHAT not how*):
  - **Functional** (what functions the SW has, what does it do)
  - **Non-functional** – what properties the SW has (security, privacy, performance, deploy in Cloud? etc.)
- **Specifications** – more concrete engineering guidance of HOW to develop the SW (often combined with requirements into SW requirement specifications - **SRS**)
- Followed by **architectural design, implementation, testing in an** ***iterative fashion***

Requirements vs. Specifications

- Distinction is blurred
- One way of thinking: requirements focus on WHAT and specifications add more of HOW information
- Often merged into Software Requirements Specification documents (SRS)
  - http://techwhirl.com/skills/tech-docs/writing-software-requirements-specs/
- SW specs: more concrete design and programming guidance (program design language, block diagrams, choice of platforms, compilers, etc,)

Important

Typical Content of a General Requirement

- Title
- Reference number (for tracking)
- Text (a few lines or a few paragraphs)
- Graphics, diagrams (optional), mandatory for UI
- Initiator, owner
- Priority e.g. 1, 2, 3 (can be assigned early or later)

Many companies have their own templates (government, military, IEEE, ACM)

Follow the required process (or work with management to improve it)

RTM (Requirements Traceability Matrix) VMware

Global Delivery Excellence

Project Requirements Traceability Matrix

Project ID: ACM000012
Customer Name: ACME
Project Manager: Tim Bennett

Keep the ID until the end

Req ID	Business Requirement or User Story	Requirement Type: Functional or Technical
SR01	Ability to utilize public networks for access to desktop image	Functional: Provide desktop image access over public networks to qualified users

Requirement / Use case. Brief.

Functional or Technical, and description

Important

Typical Content of a General Requirement - Text

- Description of service, function or constraints to be achieved
- Use case (or reference to it)
- Error conditions, special cases
- Measures to verify
- For data glossary: name, description, usage, behavior, storage

Product Requirement Documents

- Requirements are collected in a document. Early requirements are often called Marketing or Product Requirement Document (MRD or PRD). Such documents are official (approved, tracked, dates etc.).
- Use consistent format throughout the project
- Companies often have their own internal formats
- Use language in a consistent way. “Shall” means mandatory, “should” means optional.
- Avoid usage of jargon
- Data dictionary/Glossary at the beginning helps
- Keep change history: date, contributor’s name, summary of change 1-3 lines
- https://en.wikipedia.org/wiki/Product_requirements_document

Requirements document (PRD/MRD) management

- Can be made arbitrarily complex, to the point that it becomes goal in itself! Be aware!
- Make is simple to enter, modify and track
- Use established tools and standards (MS Word, Excel, HTML)
- Be careful if you want to use special tools. Problems are: cost, training, wide availability, vendor lock in
- Important to keep the versions/dates

- Again: sometimes this will be imposed on you by the company, client etc.

Important

Competitive analysis

- In design of any system one needs to know
- Goals and objectives
- Competitive landscape – who is doing something similar
- Relationship of your planned product with current market leaders
- Developed as *competitive analysis* (jointly with marketing)
- All CEOs, managers and investors ask this <u>up front</u>

Competitive analysis– know the background; who did what; and how your envision product compares

- Feature and performance comparison with competitors
- Try competitor products yourself and compare (be hones)
- Survey product review literature and analyze gaps
- Know industry trends and leverage them
- Use GUI metaphors established in relevant industry area
- Define priorities from competitive standpoint (I.e. reviewers/customers always point that competitors have this feature and we do not…)
- Work closely with marketing and sales and customer reps

Typical Competitive features table

Important

Feature	Competitor A	Competitor B	Our future product
Text Search	++	+	+
Boolean Search	+	-	+
Browse	+	+	++
Shopping cart	+	+	+

+ feature exists; ++ superior; - does not exist

Highlight your product

Requirements solicitation – gathering requirements

Important

- Requirement Solicitation/elicitation: Process of gathering requirements
- Issues
  - Scope – boundaries of the system to be built
  - Understanding – customers do not understand it either
  - Changes – requirements change in time (better undemanding, changing business/competitive environments, costs, schedules)
  - Cost effective methods of gathering – interviews, meetings, focus groups, marketing research….

http://en.wikipedia.org/wiki/Requirements_elicitation

Summerville and Sawyer, 1997

Steps in req. solicitation

Important

- Define requirement gathering methods (focus groups, meetings, interviews…).
- Identify key stakeholders and users and involve them
- Define technical environment for the system (architecture, OS, connections….)
- Business and marketing analysis
- Use case analysis
- Develop requirements:
  - Uses cases to functional specs and data glossary
  - Constrains and business/marketing issues to nonfunctional specs
- Document in Marketing Requirement Document (MRD)
  - Communicate, revise, manage MRD

REQUIREMENTS CHANGE
SO REVISE THEM AS NEEDED

Important

Issues and problems with requirements for SW

- Full requirement set is almost **never known** at early stage
- Requirements **most often change in the process** due to real reasons (people learned more, business changed, users requested it after evaluation) and some other reasons (marketing wants more, competition is there, miscommunication) ➔ recall why it is hard to manage SW projects….
- We will address this in the class (e.g. this is why Waterfall method does not work)
- This is one of the key problems in building and managing large SW systems

Important

Issues and problems with requirements (2)

- Ambiguity: can be interpreted (and will be interpreted) differently by different people or groups.
- Incompleteness: do not specify all what needs to be specified. (Authors assumed that others know too much)
- Conflicting
- Hard to verify
- Hard to use formats and document management

- It is very hard to create perfect requirements. Don't try to be perfect, be good enough for product to succeed!
- Requirements are only means to an end (I.e. successful product)

Important

Waterfall and other models that required all req. to be known ahead of the time: some history

- David Parnas, Paul Clements, 1986:
  - A system's users seldom know exactly what they want and cannot articulate all they want
  - Even if we could state all the requirements, there are many details that we can only discover once we are well into implementation
  - Even if we knew all these details, as humans, we can master only so much complexity
  - Even if we could master all this complexity, external forces lead to changes in requirements, some of which may invalidate earlier decisions

Important

Priorities

- It is critical to have priorities attached to each requirement so you can make choices when schedules and costs and resources get constrained
- Agree on priorities with marketing, executives <u>and customers</u>
- It never happens that all features are equally important
  - *Priority 1*: Critical: do not launch product without it.
  - *Priority 2*: Important: it would be really good to have it. Don't delay ship date if not available
  - *Priority 3*: Opportunistic: nice to have. Candidate for next release.
- When it comes to cutting functions/SCOPE (always happens late in the schedule) use priorities to make a decision
- Only used for functions to be provided, not for data glossary or non-functional requirements

Priorities vary from domain to domain

- *Medical*: patient and staff safety; accuracy; privacy; ease of use
- *R&D, Academic*: accuracy; performance; ease of use
- *Banking*: transaction accuracy; security; reliability; availability
- *Games*: graphics, story, usability

- Check this early and use to guide design and project management/resource optimization

Important

Priorities and specs

- Functional specs
  - Critical to prioritize
  - Priorities used to guide the design and decisions what to keep/drop/add
  - Helps plan the development (focus on priority 1 first)
  - Grouping by priorities (especially Priority 1) helps better understand what the product features are
- Nonfunctional specs
  - Usually fixed and given by funding client/management/legal/marketing
  - Generally not prioritized – all must be done
  - Can not be changed solely by engineering team

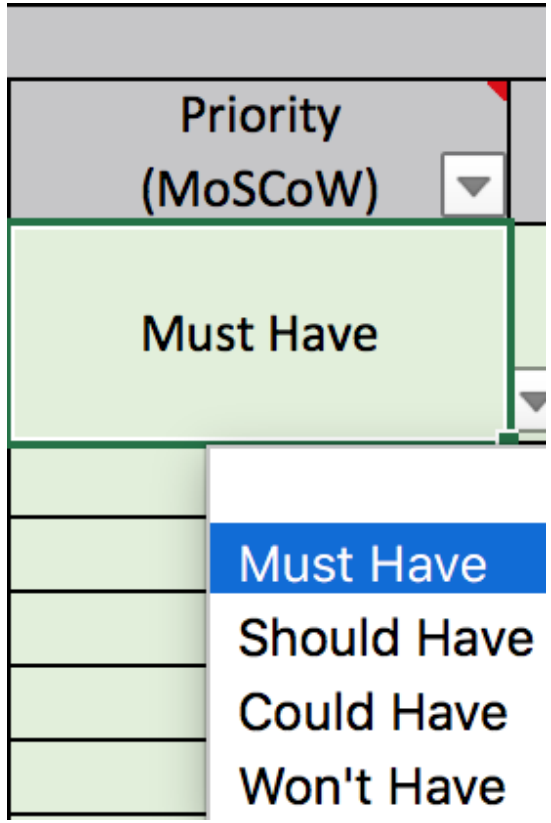
MoSCoW Priority (VMware)

Must Have
Critical to the current delivery in order for it to be a success. If one Must have is not included, the project delivery should be considered a failure

Should Have
Important but not necessary for delivery in the current stage. While Should haves can be as important as Must haves, they are often not as time-critical

Could Have
Desirable but not necessary and could improve user experience or customer satisfaction for little development cost. Included if time and resources permit.

Won't Have
The least-critical, lowest-payback items, or not appropriate at that time agreed as out of scope



Important

Requirements for UI and User-Intensive Systems

- Similar rules apply PLUS:
  - Strongly driven by use cases and types/categories of users - *personas*
  - Considerations related to u*sability, inherent characteristics of intended users and UI design principles*
  - It is very hard to communicate and specify such requirements only with text – users need to see it before they can understand or decide ➔ use of graphical mockups and storyboards is essential (with some text please)

High Level Use Cases for User Intensive Systems

Important

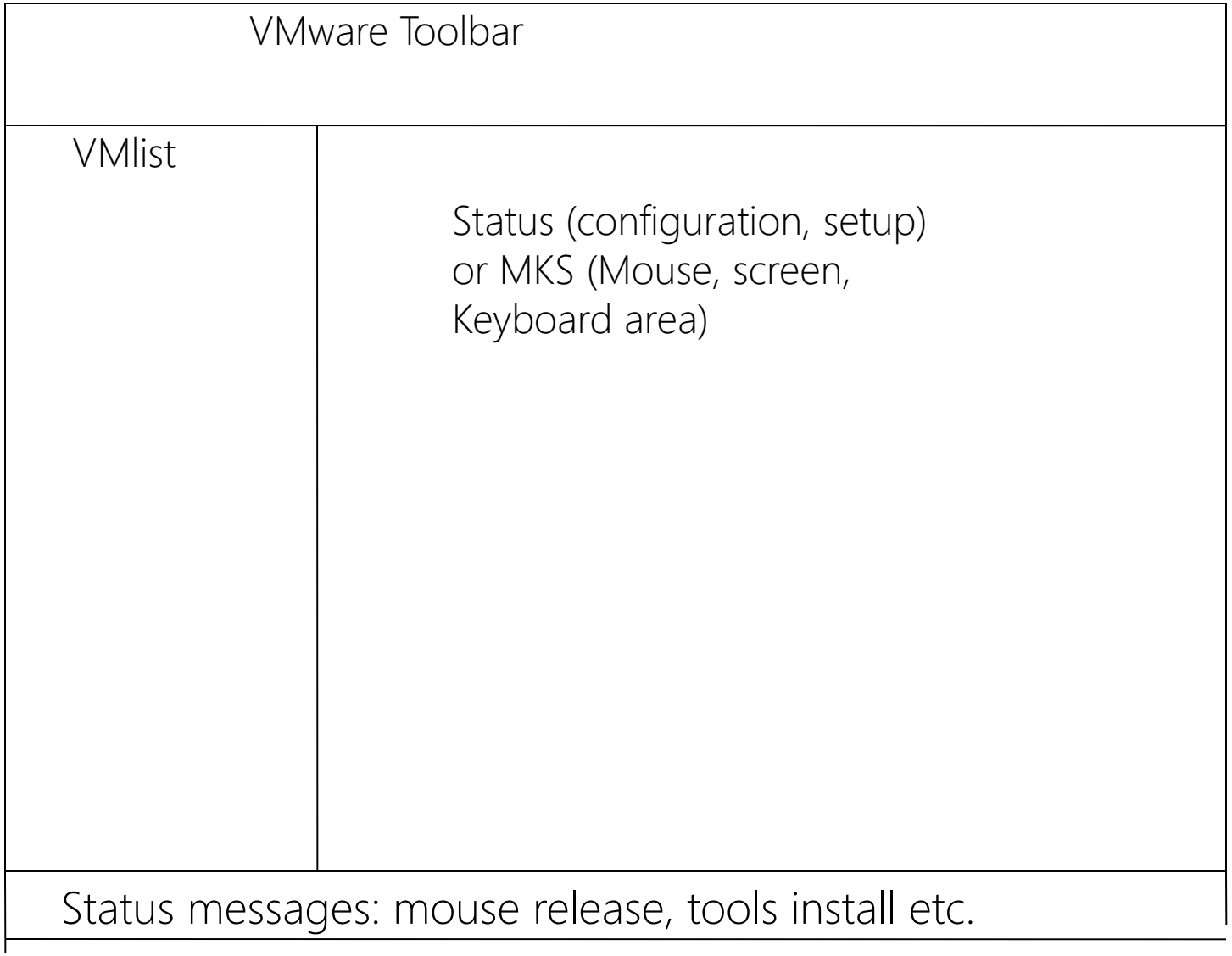
- Starting point.
- Covered before – also: important to defer specific UI decisions to later – focus on WHAT users want to do and not HOW (HOW will be done later – mockups, prototypes, feedback…)
- And also
- Summarize typical users by set of fictitious "personas" with their characteristics important for the application

Important

From Use Cases and Personas to High-level Mockups, storyboards

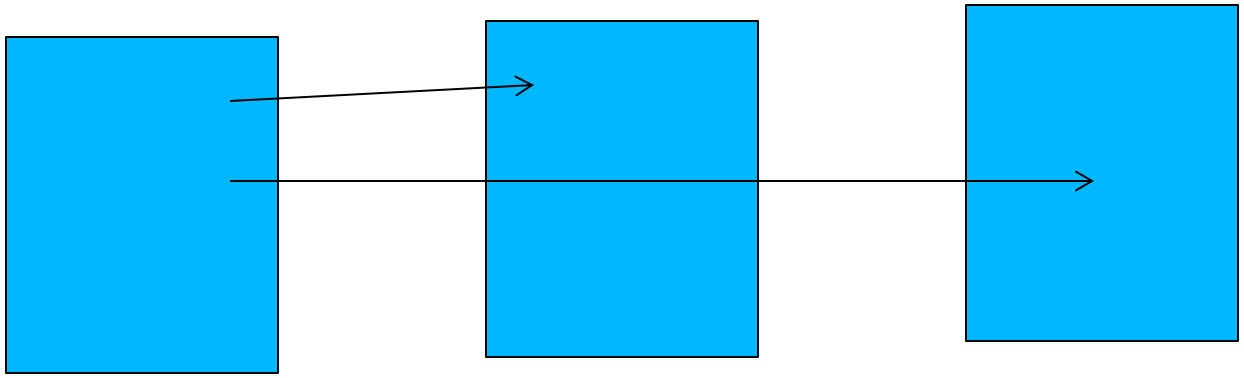
- Simplified graphical representations of envisioned UI
- Focus is on UI basics: functions and behaviors, layout, flow
- Graphics none or very minimal so as to not grab the attention
- <u>High level mockups are best B&W! (Color distracts from UX analysis)</u>
- Text description for all functions and main issues
- Format: wire diagrams, HTML, hand drawings, animation
- Mandatory for UI requirements and specifications

UI Mockups –
design of VMWare Workstation 3.0 (using “wire diagrams”)

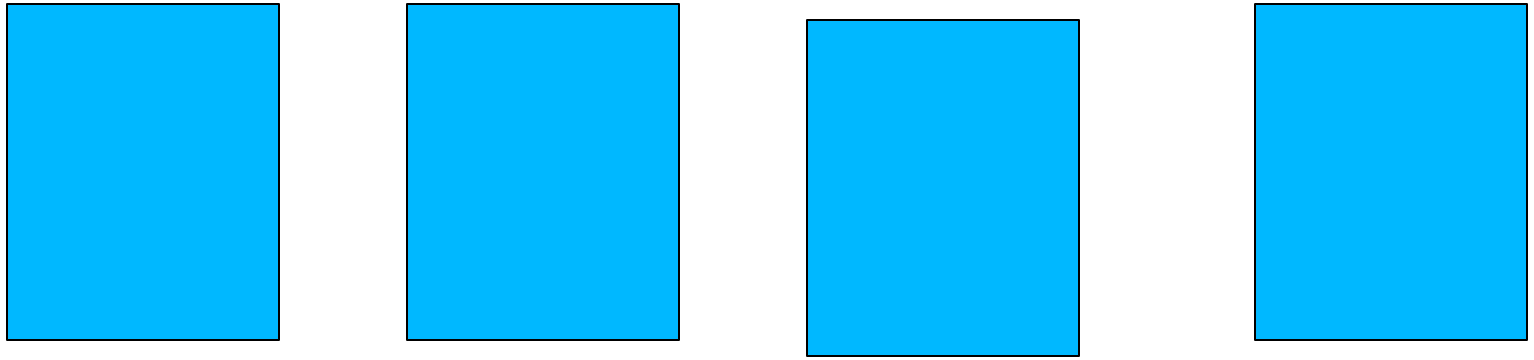


Storyboards – sequence of mockups following a use case

Use case 1



Important



Use case 2

Important

Why are simple wire diagrams best *in early stage?*

- Easy to produce (can be hand drawn too)
- Easy to change (no coding invested)
- Allow easy scribbling, markup
- No graphics or colors help focus on key issues: function and layout
- Allow considerable "scenario and usage" testing

- (Many great designs done on a napkin…)

UI Mockups and storyboards – resources (templates, examples)

- http://webdesignledger.com/inspiration/18-great-examples-of-sketched-ui-wireframes-and-mockups
- https://www.visual-paradigm.com/guide/ux-design/wireframe-vs-storyboard-vs-wireflow-vs-mockup-vs-prototyping/
- And there are tools for this
- Review – top 21
  - https://blog.prototypr.io/top-20-prototyping-tools-for-ui-and-ux-designers-2017-46d59be0b3a9
- Figma https://www.figma.com/
- Sketch https://www.sketch.com/

Important

Requirements for Media Rich apps (almost all today)

- Raw data: Files used to render content (MPEG, PDF, JPEG etc.)
- Metadata: data describing content, used for searching
- Supporting data: data used to display items in the search result list. In the form of text, but also may include key frames, samples, previews
- Price
- Copyright rules

Used to design both search as well as database organization

Requires close interaction with data owners, librarians etc.

How to develop requirements regarding content organization

Important

- This is information/data design issue not a coding issue
- Interview current owners and users
- Base it on search needs of users as well as nature of the content (common ways of describing content)
- Look at similar other products or applications
- Review legacy (current usage)
- Follow legal rules on content usage
- Application dependent:
  - Movies
  - Stock photo
  - Books
  - Shopping

Story about requirements – some wisdom and experience

- Requirements are very important, BUT be aware of perils:
  - Insisting on complete requirements is most often an obstacle to SW development. They simply may not be known at that point
  - Same holds for excessive, inflexible and cumbersome documentation and requirement process mechanics
  - "Requirement paralysis" ➔ endless discussions, while schedules are being missed
  - Break the impasse by iterative method and early prototyping, then refine
- There are applications where requirements are critical (mission critical, safety, medical, military, legal)
- Use judgment (as always) as to how much to invest in requirements process

Thank You